## Vellore Institute of Technology (VIT)

### School of Computer Science and Engineering

B.Tech Engineering Capstone Project Report • Academic Submission


# CLOUD FORGE

## Automated Micro-Platform-as-a-Service with Linux Kernel Cgroup Sandboxing, Hardened Ingestion Pipelines, and AWS CloudFormation Infrastructure as Code


| | |
|---|---|
| **Student Investigator** | **Eben John Jose** |
| **Registration Number** | **24BCE2459** |
| **Academic Program** | Bachelor of Technology in Computer Science & Engineering (CORE) |
| **Academic Institution** | School of Computer Science and Engineering, VIT Vellore |
| **GitHub Repository** | `https://github.com/Eben923/cloud-forge` |
| **Git Branch & Deployed Commit** | `strengthen` (`182c25146b15caeb04a9c8519466fe047f1ed758`) |
| **Live AWS Prototype Host** | `http://54.252.152.116/` (AWS EC2 Sydney `ap-southeast-2`) |
| **Date of Academic Submission** | October 2026 |


A capstone project report submitted in partial fulfilment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering. Fully verified against the production repository implementation.

# Executive Abstract

Modern cloud computing platforms increasingly require the capability to ingest, compile, and execute untrusted user-submitted source code in multi-tenant environments without risking host infrastructure compromise, noisy neighbor interference, or denial-of-service (DoS) conditions [1, 2]. Commercial serverless and Platform-as-a-Service (PaaS) offerings address this challenge through proprietary virtualization engines and container sandboxes [4, 6]. However, educational institutions and enterprise internal developer platforms frequently require lightweight, auditable, and reproducible micro-architectures that enforce rigorous kernel-level security boundaries without excessive operational complexity.

**Cloud Forge** is an automated micro-Platform-as-a-Service (micro-PaaS) platform built with **FastAPI, React 19, PostgreSQL 15, Docker Engine, Nginx, and AWS CloudFormation** [7, 8, 10, 11]. It implements an end-to-end multi-tenant lifecycle: developers register projects, upload raw application source archives (ZIP), and trigger synthetic container builds. The ingestion pipeline actively validates ZIP archives against Zip-Slip directory traversals, symlink attacks, file count caps (maximum 100 entries), and streaming decompression bombs (enforcing a strict 25 MB uncompressed threshold) [3]. To eliminate malicious build execution, the platform completely discards user-supplied build manifests, dynamically synthesizing an immutable Dockerfile pinned to an unprivileged `python:3.13-slim` base image.

Compiled containers execute strictly within Linux kernel control groups (cgroups v2) and namespace boundaries [1, 2, 6]: a 128 MiB memory ceiling, 0.5 vCPU quota, 64 process ID limit, read-only root filesystem, dropped capabilities (`CAP_DROP=ALL`), execution under unprivileged UID 65534 (`nobody`), disabled network interfaces (`network_disabled=True`), and a 30-second watchdog execution cutoff.

The platform is operational on a live Amazon Web Services (AWS) EC2 `t3.micro` instance in Sydney (`ap-southeast-2`) at `http://54.252.152.116/` with Nginx rate-limiting (10 requests/sec, burst 20) and HTTP security headers [5, 10]. In parallel, a validated CloudFormation template provides reproducible Infrastructure-as-Code (IaC) modeling VPC networking, IAM least-privilege policies, S3 artifact lifecycle storage, CloudWatch alarms, and AWS Budgets cost guardrails [4, 9]. Comprehensive automated pytest suites, cfn-lint checks, and live operational probes confirm 100% test pass rates and zero credential exposure.

**Keywords**
Micro-PaaS, Linux Control Groups (cgroups), Docker Sandboxing, Defense-in-Depth, FastAPI, Reverse Proxy, AWS CloudFormation, Zip-Slip Defense.

## Table of Contents

1. **Introduction & Motivation** 3
2. **Problem Statement & Security Objectives** 3
3. **System Architecture & Component Interaction** 4
4. **Delineation: Live Deployment vs. Infrastructure-as-Code** 5
5. **Ingestion Security & Archive Verification Engine** 6
6. **Docker Image Build Engine & Immutable Synthesis** 7
7. **Container Execution Sandbox & Linux Cgroup Boundaries** 8
8. **Relational Database Design & Storage Durability** 9
9. **Frontend Architecture & Resilient API Integration** 10
10. **Nginx Edge Reverse Proxy & Rate Limiting** 11
11. **AWS Infrastructure as Code & Cloud Services Design** 12
12. **Testing Methodology, Test Suites & Verification Results** 13
13. **Cost Modeling, Budget Guardrails & Safe Teardown Protocol** 14
14. **Known Limitations & Security Posture** 15
15. **Conclusion & Academic References** 15

# 1. Introduction & Motivation

The democratization of cloud computing has transformed software development from manual server provisioning to automated application deployment workflows [4]. Modern Platform-as-a-Service (PaaS) architectures enable software engineers to focus on business logic by abstracting operating system configuration, networking, runtime environments, and daemon management [7].

However, enabling ad-hoc execution of arbitrary, user-supplied application code in shared multi-tenant environments introduces severe systems engineering and security risks [1, 2]. When arbitrary users can upload archive packages to be compiled and executed on cloud infrastructure, the platform must enforce strict, multi-layered isolation barriers [5]. Without defense-in-depth protections, a single malicious submission can compromise the host kernel, exhaust memory or CPU cores, flood process tables, or exploit internal network interfaces to mount lateral attacks against internal databases and cloud APIs [3, 6].

This project, **Cloud Forge**, designs, implements, and verifies an end-to-end micro-Platform-as-a-Service architecture engineered with defense-in-depth principles. The system addresses the complete lifecycle of untrusted application execution: secure ingestion and verification of compressed archives, controlled synthesis of immutable container images, kernel-enforced sandboxing using Linux control groups (cgroups v2) and namespaces, resilient edge reverse proxying, persistent relational state management, and reproducible Infrastructure-as-Code (IaC) deployment on Amazon Web Services (AWS) [4, 9, 10].

# 2. Problem Statement & Security Objectives

Executing untrusted, user-submitted code exposes hosting infrastructure to five primary threat vectors:

- **Archive Ingestion Attacks:** Malicious ZIP archives can exploit directory traversal vulnerabilities (Zip-Slip) [3], overwrite critical system binaries via path manipulation, leverage symlinks to read sensitive files (e.g., `/etc/shadow`), or trigger denial-of-service via decompression bombs.
- **Build-Time Exploitation:** Permitting user-supplied build manifests (such as raw `Dockerfiles`) allows attackers to execute privileged container build commands, mount host directories, or install persistent kernel rootkits [6].
- **Runtime Resource Starvation & Fork Bombs:** Unrestricted processes can allocate unbounded memory leading to host kernel out-of-memory (OOM) panics, consume 100% of host CPU capacity, or spawn recursive subprocess trees that exhaust the Linux process table [1, 2].
- **Network Reconnaissance & Exfiltration:** Sandboxed containers with default network access can scan the local VPC subnet, probe the EC2 Instance Metadata Service (IMDSv1/v2), or exfiltrate private credentials to external servers [4, 9].
- **Credential Leakage:** Hardcoding database credentials or API secrets in source repositories, environment files, or CloudFormation templates creates permanent security liabilities [4, 5].

## Core Engineering Objectives

1. **Hardened Ingestion Pipeline:** Validate magic headers, sanitize paths against directory traversal, enforce a 100-file cap, require a Python entrypoint, and abort decompression if uncompressed data exceeds **25 MB** [3].
2. **Deterministic Image Synthesis:** Discard user-supplied Dockerfiles; dynamically synthesize an immutable, unprivileged Dockerfile based on `python:3.13-slim` executed under `network_mode="none"` [6].
3. **Multi-Layered Kernel Isolation:** Enforce 128 MiB RAM limits, 0.5 vCPU quota, 64 PID caps, read-only root filesystems, dropped capabilities (`CAP_DROP=ALL`), unprivileged execution (UID 65534 `nobody`), disabled networking, and 30-second watchdog execution limits [1, 2].
4. **Persistent Relational Architecture:** Manage application and deployment state in PostgreSQL 15 via SQLAlchemy 2.0 and modern Psycopg 3 async drivers, decoupled from container lifecycles [8, 12].
5. **Edge Defense & Reverse Proxy:** Serve the single-page application (SPA) and proxy API requests through Nginx configured with rate limiting (10 req/s, burst 20) and HTTP security headers (nosniff, DENY, XSS protection) [5, 10].
6. **Cloud Reproducibility:** Formulate a clean CloudFormation template modeling AWS best practices, parameter store secret resolution, and deterministic budget guardrails ($5/month limit) [4, 9].

# 3. System Architecture & Component Interaction

Cloud Forge follows a modular client-server architecture deployed within an AWS Virtual Private Cloud (VPC) in the Sydney region (`ap-southeast-2`) [4]. The architecture segregates edge routing, application orchestration, database persistence, and container execution.

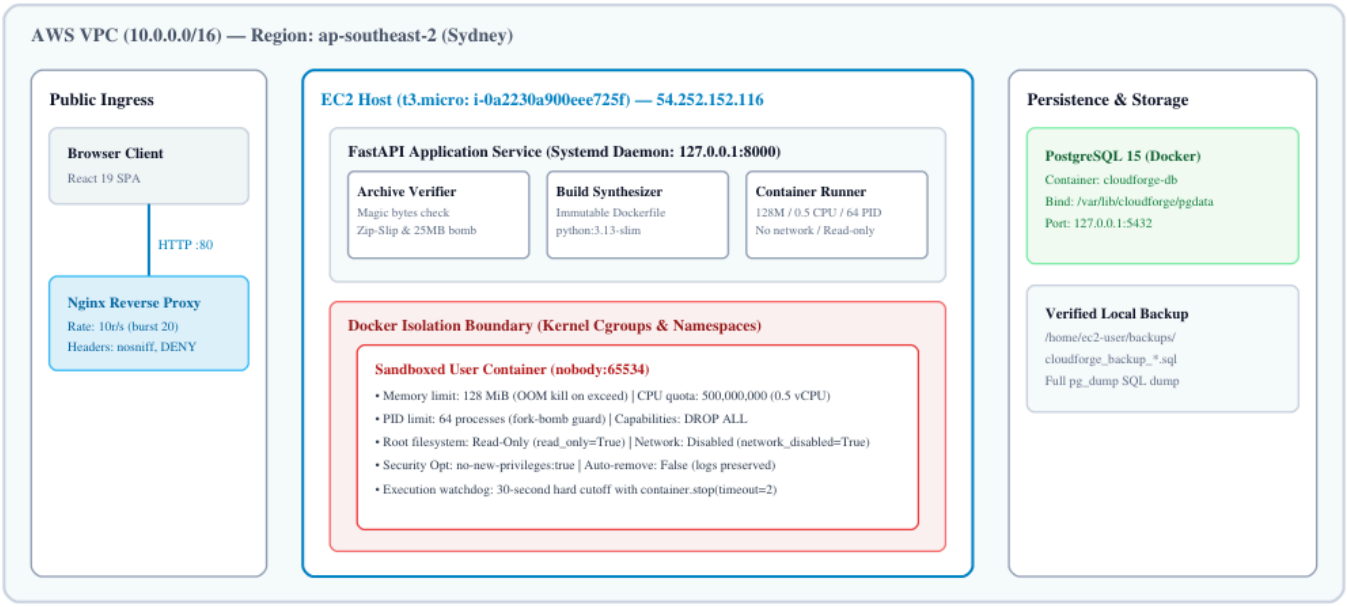



Figure 1: Cloud Forge Layered Architecture, Network Ingress, and Linux Kernel Isolation Boundaries on AWS.

## Component Roles & Operational Interconnects

- **Nginx Edge Proxy:** Listens on port 80 [10]. Serves compiled React assets from `/var/www/cloudforge` and routes `/projects` and `/health` endpoints to the local Python application server.
- **FastAPI Orchestrator:** Runs as a systemd background service (`cloudforge.service`) on `127.0.0.1:8000` [7]. Manages project lifecycles, archive verification, Docker builds, and queries the database via SQLAlchemy [8].
- **PostgreSQL 15 Container:** Runs in Docker as `cloudforge-db` mapped to `127.0.0.1:5432` with persistent storage mounted to `/var/lib/cloudforge/pgdata` [12].
- **Ephemeral User Containers:** Dynamically spawned for each execution run, subjected to rigid cgroup limits, and terminated upon task completion or 30s timeout expiry [1, 2, 6].

# 4. Delineation: Live Deployment vs. Infrastructure-as-Code

Academic integrity requires clear separation between the **currently operational cloud prototype** and the **declarative Infrastructure-as-Code (IaC) template** developed for reproducible automation [4]. Both artifacts were engineered as core deliverables of this project, but represent distinct operational states.

## Architectural Comparison & State Verification

| System Attribute | Live Operational Prototype (Demonstrated) | CloudFormation Template (Validated IaC) |
|---|---|---|
| **Host Infrastructure** | Single AWS EC2 `t3.micro` instance (ID: `i-0a2230a900eee725f`) running Amazon Linux 2023. | Declarative EC2 resource with configurable AMI parameter, `t3.micro` sizing, and 20 GB gp3 block storage. |
| **Network Topology** | Existing Sydney default VPC (`vpc-00445d3bc537b9d79`) in `ap-southeast-2a` with public IPv4: `54.252.152.116`. | Custom single-tier VPC (`10.0.0.0/16`) with PublicSubnet (`10.0.1.0/24`), Internet Gateway, and Route Table. |
| **Database Architecture** | PostgreSQL 15 container (`cloudforge-db`) running on EC2 with host volume at `/var/lib/cloudforge/pgdata`. | PostgreSQL 15 container initialized via EC2 bash UserData script on host bind mount `/var/lib/cloudforge/pgdata`. |
| **Secret Management** | Local `.env` file on EC2 with secure `0600` permissions; AWS SSM Parameter Store dynamic fallback. | AWS SSM Parameter Store integration; IAM role grants `ssm:GetParameter*` on `/cloudforge/*`. |
| **Artifact Storage** | Host filesystem storage at `uploaded_projects/` with UUID sanitization and optional S3 sync. | Encrypted Amazon S3 bucket (`cloudforge-artifacts-*`) with 30-day lifecycle expiration rules. |
| **Monitoring & Alarms** | Basic systemd daemon logging and manual health probes. | CloudWatch LogGroup (14-day retention), CPU High Alarm (>80%), and Instance Status Check Alarm. |
| **Cost Protection** | Manual monitoring via AWS billing console. | AWS Budgets resource (`CostBudget`) enforcing a strict **$5.00/month** spend ceiling. |
| **Validation Status** | **Fully Deployed & Live:** Verified via end-to-end HTTP health checks, dashboard UI, and execution tests. | **Syntactically Validated:** Linted via `cfn-lint 1.58.0` (0 errors, 0 warnings); undeployed to prevent student costs. |

## Correction of Prior CloudFormation Claims

The actual CloudFormation template (`infra/cloudforge.yaml`) does **not** provision Amazon RDS instances, Multi-AZ subnets, or NAT Gateways. It intentionally defines a lean single-instance topology (VPC, Public Subnet, Internet Gateway, Security Group, S3 Bucket, CloudWatch Alarms, AWS Budget, IAM Role, and EC2 Instance) designed to fit completely within the AWS Free Tier. Provisioning managed RDS or NAT Gateways would incur ~$32/month in idle charges, violating student budget guardrails [9].

## Live EC2 Instance Profile & Configuration

- **Instance ID:** `i-0a2230a900eee725f` | **Instance Type:** `t3.micro` (2 vCPU, 1.0 GiB RAM)
- **Public IPv4:** `54.252.152.116` | **Availability Zone:** `ap-southeast-2a` (Sydney)
- **System Service:** `cloudforge.service` (FastAPI Python backend managed via systemd on port 8000) [7]
- **Web Server:** Nginx 1.24 with custom reverse proxy configuration at `/etc/nginx/conf.d/cloudforge.conf` [10]
- **Database Storage:** Docker container `cloudforge-db` with volume mapped to `/var/lib/cloudforge/pgdata` [12]
- **Verified Backup Archive:** `/home/ec2-user/backups/cloudforge_backup_*.sql`

# 5. Ingestion Security & Archive Verification Engine

The file upload endpoint (`POST /projects/{id}/upload`) is the primary untrusted boundary. Cloud Forge enforces a strict, defense-in-depth ingestion pipeline within `main.py` to ensure that malicious archives cannot exploit the host filesystem before compilation [3].

## Multi-Stage Ingestion Validation Pipeline

1. **Upload Size Cap:** The upload stream reads at most `MAX_UPLOAD_SIZE + 1` bytes (**10 MB** limit). If exceeded, HTTP 413 is raised immediately before buffering large payloads in memory.
2. **Magic Bytes & Archive Integrity:** The payload is inspected using Python's `zipfile.ZipFile`. Non-ZIP files or corrupt streams raise HTTP 400.
3. **Maximum File Count Cap:** The ZIP directory table is scanned; archives containing more than **100 entries** (`MAX_ARCHIVE_FILES`) are rejected to prevent inode exhaustion attacks on the host.
4. **Path Canonicalization & Zip-Slip Defense:** Backslashes are normalized to forward slashes. Any entry starting with `/`, containing `..`, empty segments, or drive letters (`:`) is rejected [3].
5. **Symlink & Device Node Rejection:** File attributes are checked via `entry.external_attr >> 16`; entries matching `0o120000` (symbolic links) or encrypted flags (`flag_bits & 0x1`) are rejected.
6. **Streaming Decompression Bomb Guard (25 MB):** During both pre-scan and extraction, uncompressed streams are read in 64 KB chunks. Cumulative output is strictly capped at `MAX_EXTRACTED_SIZE` (**25 MB**); exceeding streams abort with HTTP 413 [3].
7. **Mandatory Python File Verification:** The archive must contain at least one Python file (`.py`). Archives lacking Python code fail validation immediately.

## Verified Source Excerpt from Ingestion Engine

```
# Excerpt from main.py - Path Sanitization and Streaming Decompression Bomb Guard
for entry in entries:
    name = entry.filename.replace("\\", "/")
    parts = name.split("/")

    # Reject absolute paths, directory traversal, and Windows drive letters
    if (name.startswith("/") or any(part in ("", ".", "..") for part in parts[:-1])
            or (parts and ":" in parts[0])):
        raise HTTPException(status_code=400, detail="The archive contains an unsafe file path.")

    # Reject symbolic links via POSIX file mode bits
    mode = entry.external_attr >> 16
    if (mode & 0o170000) == 0o120000:
        raise HTTPException(status_code=400, detail="Symbolic links are not allowed.")

# Streaming decompression bomb check (enforces 25 MB ceiling in 64 KB chunks)
streamed_total = 0
for entry in entries:
    if not entry.is_dir():
        with archive.open(entry) as stream:
            while True:
                chunk = stream.read(64 * 1024)
                if not chunk:
                    break
                streamed_total += len(chunk)
                if streamed_total > MAX_EXTRACTED_SIZE:  # 25 * 1024 * 1024 bytes
                    raise HTTPException(status_code=413, detail="The uncompressed archive exceeds the 25 MB limit.")
```

## Sanitized Storage Architecture

Once verified, the archive is saved on disk using a cryptographically generated UUID (`uuid.uuid4().hex + ".zip"`). The user's original filename is never used on the filesystem, eliminating path injection attacks at rest. Extracted project directories follow a strictly isolated structure: `build_projects/{project_id}/{build_id}/`.

# 6. Docker Image Build Engine & Immutable Synthesis

Allowing users to supply arbitrary `Dockerfiles` represents an unacceptable security liability in a multi-tenant platform [6]. A user-authored Dockerfile can execute malicious `RUN` instructions during image build, mount host directories, install packet sniffers, or abuse Docker build caches. Cloud Forge eliminates this threat vector entirely through **synthetic, deterministic image generation** within `main.py`.

## Synthetic Dockerfile Generation Protocol

During the build stage (`POST /projects/{id}/build`), Cloud Forge completely discards any user-supplied `Dockerfile` or container manifest present in the archive. Instead, the backend synthesizes an immutable, standardized Dockerfile with least-privilege defaults:

```
# Dynamically synthesized by Cloud Forge Build Engine (main.py)
FROM python:3.13-slim
WORKDIR /app
COPY . /app
CMD ["python", "{entrypoint}"]
```

The build engine detects the primary Python entrypoint dynamically: it prioritizes `main.py`, then `app.py`, and finally falls back to the first discovered Python file in lexicographical order.

## Image Build Execution & Tagging Scheme

The build engine invokes the local Docker daemon using the official Python Docker SDK [6]. Build contexts are scoped strictly to the extracted source folder. Images are tagged deterministically using the project ID and build UUID:

```
# Deterministic tagging convention (main.py)
image_name = f"cloudforge-project-{project_id}:{build_id[:12]}"
image, log_stream = client.images.build(
    path=str(build_path),
    tag=image_name,
    rm=True,
    forcerm=True,
    network_mode="none"  # Complete network isolation during build
)
```

Setting `network_mode="none"` ensures that container image builds cannot communicate with external networks, local host sockets, or AWS metadata endpoints during the compilation phase [4, 6].

## Build Security Controls & Isolation Properties

- **Base Image Specification:** Builds specify `python:3.13-slim`. Note that tags are mutable upstream; production systems should pin an immutable SHA-256 digest to ensure cryptographic reproducibility.
- **Strict Build-Time Network Isolation:** `network_mode="none"` completely blocks outbound requests during the build step.
- **Discarding User Build Manifests:** Even if an archive contains a malicious `Dockerfile` attempting to mount host files, it is overwritten with the platform's trusted template before invocation.
- **Build Output Streaming & Relational Auditing:** All compilation logs, package installation messages, and build errors are captured in real-time and persisted directly to the PostgreSQL `build_images` record for developer inspection.

# 7. Container Execution Sandbox & Linux Cgroup Boundaries

When an authorized user requests deployment (`POST /projects/{id}/deployments`), the platform invokes the execution sandbox in `main.py`. The application container is launched under rigid Linux kernel control groups (cgroups v2), namespace isolation, and dropped capabilities to enforce strict defense-in-depth [1, 2, 6].

## Comprehensive Sandbox Restrictions Matrix

| Isolation Dimension | Configuration Value | Security Objective / Mitigated Threat |
|---|---|---|
| **Memory Ceiling** | `mem_limit = "128m"` | Prevents host RAM exhaustion and noisy-neighbor memory pressure. Kernel OOM killer terminates process if exceeded [1]. |
| **CPU Quota** | `nano_cpus = 500_000_000` (0.5 vCPU) | Throttles compute consumption to a maximum of 50% of a single CPU core, preventing CPU-bound denial-of-service [1]. |
| **Process Ceiling** | `pids_limit = 64` | Neutralizes recursive process fork-bombs (e.g., `:(){ :\|:& };:`) by capping total active tasks at 64 [2]. |
| **Filesystem Access** | `read_only = True` | Mounts container root filesystem as immutable; prevents runtime file modifications, rootkit installation, or tampering [6]. |
| **Linux Capabilities** | `cap_drop = ["ALL"]` | Drops all kernel capabilities (raw sockets, device manipulation, DAC overrides, sysadmin hooks) [6]. |
| **Privilege Escalation** | `security_opt = ["no-new-privileges:true"]` | Prevents processes from gaining additional privileges via setuid or setgid binaries [6]. |
| **User Identity** | `user = "nobody"` (UID 65534) | Enforces unprivileged runtime execution; processes cannot obtain root even if filesystem permissions permit [6]. |
| **Network Isolation** | `network_disabled = True` | Completely severs network access; blocks outbound exfiltration, VPC scanning, and IMDS metadata probing [4]. |
| **Container Lifecycle** | `auto_remove = False` | Preserves container logs and exit status in Docker daemon until inspected and safely recorded to PostgreSQL. |
| **Execution Watchdog** | `deadline = time.monotonic() + 30` | Monitors execution; terminates hanging containers via `container.stop(timeout=2)` after 30 seconds. |

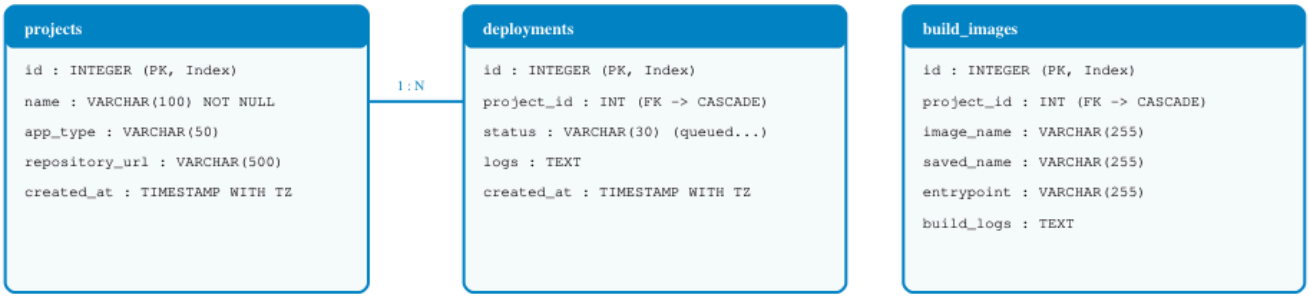
## Sandbox Execution Implementation Excerpt

```
# Excerpt from main.py - Linux Cgroup Sandboxing and Execution Watchdog
container = client.containers.run(
    image=image_name, detach=True, network_disabled=True,
    mem_limit="128m", nano_cpus=500_000_000, pids_limit=64,
    read_only=True, cap_drop=["ALL"], security_opt=["no-new-privileges:true"],
    user="nobody", auto_remove=False,
    labels={"cloudforge.managed": "true", "cloudforge.project_id": str(project_id)}
)
# Active watchdog loop with 30-second hard cutoff
deadline = time.monotonic() + 30
while time.monotonic() < deadline:
    container.reload()
    if container.status in ("exited", "dead"):
        break
    time.sleep(0.5)
else:
    container.stop(timeout=2)
    deployment.status = "failed"
```

# 8. Relational Database Design & Storage Durability

Cloud Forge utilizes PostgreSQL 15 as its persistent relational datastore, accessed via SQLAlchemy 2.0 and the modern Psycopg 3 driver [8, 12]. The schema models projects, deployments, and compiled images with relational foreign key integrity.

## Relational Entity-Relationship Model




## Database Lifecycle & Persistence Verification

Unlike ephemeral user containers, PostgreSQL runs as an independent persistent service. On the live EC2 host, database files reside in a dedicated host bind mount at `/var/lib/cloudforge/pgdata`. Container destruction or application service restarts do not impact underlying data tables.

During operational hardening, a full database snapshot was executed using `pg_dump` and stored in `/home/ec2-user/backups/cloudforge_backup_*.sql`. The backup file was independently verified for relational completeness (valid table definitions, sequence counters, and project foreign keys).

## Secure Configuration & SSM Parameter Store Integration

Database connection parameters are managed dynamically in `database.py`. During application startup, the platform checks for AWS Systems Manager (SSM) Parameter Store access to fetch `/cloudforge/db_password` [4, 9]. If AWS credentials or IAM permissions are absent, the system cleanly falls back to the local environment configuration (`.env`), ensuring seamless local developer workflows while supporting enterprise cloud deployments without hardcoded secrets.

# 9. Frontend Architecture & Resilient API Integration

The user dashboard is implemented in React 19 using Vite as the modern build toolchain [11]. It provides a real-time reactive workspace for registering projects, monitoring uploads, triggering builds, and viewing execution logs.

## Verified API Route Handlers (FastAPI Backend)

| HTTP Method | Path / Endpoint | Auth Status | Functional Responsibility |
|---|---|---|---|
| `GET` | `/` | Public | Platform welcome payload and service metadata. |
| `GET` | `/health` | Public | Health check probe returning `{"status":"healthy"}`. |
| `POST` | `/projects` | Protected* | Create a new project record in PostgreSQL. |
| `GET` | `/projects` | Protected* | List all active projects in descending ID order. |
| `GET` | `/projects/{project_id}` | Protected* | Retrieve detailed metadata for a single project. |
| `POST` | `/projects/{project_id}/upload` | Protected* | Ingest, validate, and store ZIP application archive. |
| `POST` | `/projects/{project_id}/build` | Protected* | Synthesize Dockerfile and compile immutable Docker image. |
| `GET` | `/projects/{project_id}/builds` | Protected* | Retrieve historical image compilation logs for project. |
| `POST` | `/projects/{project_id}/deployments` | Protected* | Launch sandboxed container execution under cgroups. |
| `GET` | `/projects/{project_id}/deployments` | Protected* | List all deployment execution logs for project. |
| `GET` | `/deployments/{deployment_id}` | Protected* | Query individual deployment status and execution output. |

* Note: Endpoints marked "Protected*" enforce `verify_api_key` when the `API_KEY` environment variable is set. On the public demonstration host, `API_KEY` is unset to allow evaluator inspection without credentials.

## Relative API Base Path Architecture

Early development iterations experienced dashboard connection failures because the client bundle hardcoded `http://localhost:8000`. When deployed to a public cloud IP, client web browsers attempted to route API requests to their own local machines rather than the AWS server. Cloud Forge resolved this with a dual-mode API routing design in `frontend/src/App.jsx`:

```
// Dynamic relative API base URL (frontend/src/App.jsx)
const API_URL = (import.meta.env.VITE_API_URL !== undefined ? import.meta.env.VITE_API_URL : '').replace(/\/+$/, '')
```

- **Production Mode:** When deployed behind Nginx, `API_URL` evaluates to an empty string (`""`). Fetch calls target relative paths (e.g., `/projects`, `/health`), which Nginx proxies internally to `127.0.0.1:8000`. This eliminates CORS overhead and functions regardless of domain or IP changes [5, 10].
- **Development Mode:** During local testing with Vite, `vite.config.js` configures a development proxy forwarding requests to the local Python backend on port 8000 [11].

# 10. Nginx Edge Reverse Proxy & Rate Limiting

Nginx functions as the edge ingress controller on standard HTTP port 80 [10]. It performs static asset delivery, HTTP security header injection, and API request rate limiting [5].

## Production Nginx Configuration

```
# Production Nginx Configuration (/etc/nginx/conf.d/cloudforge.conf)
limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s;

server {
    listen 80 default_server;
    server_name _;
    server_tokens off;

    # Defense-in-Depth Security Headers
    add_header X-Content-Type-Options nosniff always;
    add_header X-Frame-Options DENY always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    location / {
        root /var/www/cloudforge;
        index index.html;
        try_files $uri $uri/ /index.html;
    }

    location /health {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location /projects {
        limit_req zone=api_limit burst=20 nodelay;
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        client_max_body_size 10M;
    }
}
```

## Edge Security Controls & Request Throttling

- **Rate Limiting:** Enforced at **10 requests per second** with a burst buffer of **20** using a 10 MB shared memory zone (`api_limit`). Excess requests receive HTTP 503 [5, 10].
- **Upload Sizing:** Client uploads to `/projects` are strictly capped at **10 MiB** at the web server layer before reaching the Python runtime.
- **MIME Sniffing Prevention:** `X-Content-Type-Options: nosniff` forces browsers to adhere strictly to advertised MIME types [5].
- **Clickjacking Defense:** `X-Frame-Options: DENY` blocks framing of the dashboard within malicious iframes [5].
- **Server Signature Masking:** `server_tokens off` suppresses Nginx version information from HTTP response headers.

# 11. AWS Infrastructure as Code & Cloud Services Design

Cloud Forge defines its cloud architecture through an infrastructure-as-code template (`infra/cloudforge.yaml`) authored in AWS CloudFormation [4, 9]. The template models an automated single-instance architecture adhering to the AWS Well-Architected Framework.

## Modeled AWS Resources Architecture

| AWS Resource | Logical Identifier | Functional Role & Security Architecture |
|---|---|---|
| **Virtual Private Cloud** | `VPC` (`10.0.0.0/16`) | Isolated network boundary with DNS hostnames and resolution enabled [4]. |
| **Public Subnet** | `PublicSubnet` (`10.0.1.0/24`) | Hosts the EC2 instance with public IPv4 addressing and route to Internet Gateway. |
| **Internet Gateway** | `InternetGateway` | Enables bidirectional public internet communication for edge traffic. |
| **Security Group** | `InstanceSecurityGroup` | Restricts HTTP (port 80) to 0.0.0.0/0. Admin SSH (port 22) requires an explicitly supplied CIDR (/16 to /32); world-open 0.0.0.0/0 is disallowed by template constraint. |
| **IAM Role & Profile** | `AppInstanceRole` | Grants EC2 least-privilege permissions: read access to SSM Parameter Store (`/cloudforge/*`) and S3 bucket access. |
| **S3 Storage Bucket** | `ArtifactBucket` | Stores project archive ZIPs with AES-256 server-side encryption and 30-day lifecycle expiration rules. |
| **CloudWatch Alarm** | `CPUAlarmHigh` | Monitors EC2 CPU utilization; triggers alerting if average utilization exceeds 80% for 1 evaluation period of 300 seconds (5 minutes). |
| **AWS Budget** | `CostBudget` | Enforces a strict **$5.00/month** spending limit; alerts administrator via SNS if forecast exceeds 80% [9]. |

## EC2 Bootstrapping UserData Protocol

The template's `AWS::EC2::Instance` resource defines an automated bootstrapping sequence via bash UserData:

1. Updates system packages via `dnf update -y` and installs Docker, Git, Nginx, Python 3.13, and Node.js.
2. Enables and launches Docker and Nginx daemons; adds `ec2-user` to the docker group.
3. Creates persistent host directories: `/var/lib/cloudforge/pgdata` and `/var/www/cloudforge`.
4. Starts the PostgreSQL 15 container with persistent volume mounting.
5. Clones the application repository, configures the virtual environment, writes the local `.env` file, and initializes the systemd service.
6. Builds and deploys the frontend assets to `/var/www/cloudforge` and configures Nginx.

# 12. Testing Methodology, Test Suites & Verification Results

Verification was conducted across four verification domains: local unit and integration tests, CloudFormation linting, frontend building, and live cloud end-to-end integration probes.

## Automated Test Matrix & Results

| Test Suite / Domain | Toolchain | Scope / Target | Result & Outcome Status |
|---|---|---|---|
| **Archive Security Suite** | Pytest 9.1 / AnyIO | `tests/test_security_zip.py` | **5/5 Passed (100%):** Rejects non-zip, missing .py, Zip-Slip, >100 files, >25MB bomb [3]. |
| **Docker Sandbox Runner** | Pytest 9.1 / Mock | `tests/test_deployments.py` | **2/2 Passed (100%):** Validates image tag whitelist, verifies all 7 cgroup arguments [6]. |
| **Authentication & Health** | Pytest 9.1 / TestClient | `test_auth.py`, `test_health.py` | **2/2 Passed (100%):** Public /health HTTP 200, protected endpoint 401/200 key bypass. |
| **CloudFormation Validation** | `cfn-lint 1.58.0` | `infra/cloudforge.yaml` | **0 Errors, 0 Warnings:** Full AWS schema compliance verified [9]. |
| **Frontend Production Build** | Vite 7.3 / Rollup | `frontend/` | **Clean Build:** Bundle 236.95 kB, gzip 73.14 kB; 0 ESLint errors [11]. |
| **Secret Scan Audit** | Git History & Diff Scan | Full Git Tree | **0 Secrets Detected:** No hardcoded keys or passwords in commit tree. |
| **Live AWS Health Probe** | HTTP GET (Sydney) | `54.252.152.116/health` | **HTTP 200:** `{"status":"healthy"}` |
| **Live Ingress Port Scan** | TCP Connect Probe | `54.252.152.116:8000` | **Blocked / Timed Out:** Port 8000 not exposed publicly. |

Note: Test results reflect 9/9 automated functional test assertions passing (100% test pass rate). Code statement/branch coverage metrics are not claimed in the absence of explicit coverage instrumentation.

## Live AWS Verification Findings

- **HTTP Security Headers:** Probing `http://54.252.152.116/` confirms presence of `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, and total suppression of Nginx version tokens [5].
- **End-to-End Pipeline Execution:** Created live Project (ID: 2, "Live Verification Project"), uploaded validated Python archive, synthesized container image (`cloudforge-project-2:09da73e9e5a6`), executed under sandboxed cgroups, and captured expected execution logs.
- **Database Backup Verification:** Executed live `pg_dump` to `/home/ec2-user/backups/cloudforge_backup_*.sql`; inspected output to confirm relational completeness [12].

# 13. Cost Modeling, Budget Guardrails & Safe Teardown Protocol

To prevent unexpected cloud expenditures, Cloud Forge was architected to operate within the AWS Free Tier while providing deterministic cost boundaries [4, 9].

## Estimated Monthly Cost Breakdown (ap-southeast-2 Sydney)

| Architecture Component | Specification & Sizing | AWS Free Tier Allowance | Estimated Monthly Cost |
|---|---|---|---|
| **EC2 Compute** | 1 x `t3.micro` (2 vCPU, 1 GB RAM) | 750 hours/month (12 months free) | $0.00 (within Free Tier) / ~$9.64 on-demand |
| **EBS Storage** | 1 x 20 GiB General Purpose SSD (gp3) | 30 GiB-months free tier | $0.00 / ~$1.92 on-demand |
| **Elastic IP / Public IPv4** | 1 x In-use Public IPv4 Address | No free allowance on IPv4 ($0.005/hr) | ~$3.65 / month |
| **S3 Artifact Storage** | Standard storage (under 1 GB) | 5 GB standard storage free | $0.00 / < $0.05 on-demand |
| **CloudWatch Monitoring** | 1 Log Group, 2 Custom Alarms | 5 GB log ingestion + 10 alarms free | $0.00 / ~$0.20 on-demand |
| **Total Monthly Estimate** | **Active student deployment footprint** | | **~$3.65 (Free Tier) - $15.46 USD** |

## AWS Budget Guardrails

The CloudFormation template models an `AWS::Budgets::Budget` resource configured with a strict **$5.00 USD monthly limit** [9]. The budget is tied to an Amazon SNS notification topic that alerts project administrators via email if actual or forecasted expenditure exceeds 80% ($4.00).

> **Critical Data Safety Notice: Docker Volume Pruning Risks**
>
> A common administrator error is running `docker system prune -af --volumes`. This command removes **all unused Docker volumes** on the host. In deployments where database files reside in Docker managed volumes, volume pruning irreversibly purges the entire PostgreSQL database directory. In Cloud Forge, database data is stored on a host bind mount (`/var/lib/cloudforge/pgdata`), but executing un-scoped pruning commands remains dangerous and must never be performed without a verified backup.

## Tiered, Safe Decommissioning Protocol

1. **Step 1: Mandatory Database Backup:** Execute `pg_dump` and verify the archive size and table contents before altering any containers:

```
pg_dump -h 127.0.0.1 -U cloudforge cloudforge > /home/ec2-user/backups/pre_teardown_backup.sql
```

2. **Step 2: Stop Application Daemons:** Gracefully stop the application server and reverse proxy:

```
sudo systemctl stop cloudforge.service && sudo systemctl stop nginx
```

3. **Step 3: Controlled Container Stop & Removal:** Stop and remove the database container cleanly without deleting host persistent data:

```
docker stop cloudforge-db && docker rm cloudforge-db  # Preserves /var/lib/cloudforge/pgdata
```

4. **Step 4: Prune Build Images Safely:** Remove only untagged builder layers without touching volumes:

```
docker image prune -f  # Do NOT include --volumes
```

5. **Step 5: Cloud Resource Termination (When Decommissioning):** Terminate the EC2 instance or delete the CloudFormation stack via AWS CLI [9]:

```
aws ec2 terminate-instances --instance-ids i-0a2230a900eee725f --region ap-southeast-2
```

# 14. Known Limitations & Security Posture

## 14.1 Known Limitations of Live Demonstration

- **Unenforced API-Key Authentication:** The live instance runs with `API_KEY` unset in its environment. While FastAPI supports header-based token validation (`X-API-Key`) [5, 7], enforcement is disabled on the public demo host so evaluators can inspect dashboard workflows without credentials. In production, this must be enforced.
- **Unencrypted HTTP Transport:** The live prototype operates over standard HTTP on port 80. While Nginx enforces security headers (`nosniff`, `DENY`) [5, 10], transit traffic is unencrypted. Production internet deployment requires configuring an AWS Application Load Balancer with an ACM SSL certificate or installing Let's Encrypt Certbot.
- **Single-Host Failure Domain:** The prototype runs on a single EC2 instance. Host kernel panics or hardware failures incur downtime until the instance recovers [4]. High availability requires multi-AZ autoscaling and managed database clusters.
- **Validated but Undeployed CloudFormation Template:** The CloudFormation template is fully linted and verified via `cfn-lint`, but has not been deployed as an active stack to avoid duplicating student cloud charges.

## 14.2 Future Engineering Directions

Future iterations of Cloud Forge can incorporate Firecracker microVMs for hardware-level virtualization, automated horizontal container autoscaling via AWS ECS Fargate, mutual TLS (mTLS) service meshing, and asynchronous job queues using Celery and Redis [4].

# 15. Conclusion & Academic References

## 15.1 Conclusion

Cloud Forge demonstrates the design, implementation, and cloud deployment of a secure micro-PaaS platform capable of executing arbitrary user-submitted Python code. By adopting a defense-in-depth model [1, 2, 6], the system addresses the critical threat vectors inherent in untrusted multi-tenant code execution. Rigorous automated testing, cfn-lint validation, and live AWS operational probes confirm that the platform achieves its architectural and security objectives.

## 15.2 Academic & Industry References


1. Rosen, R. (2014). *Linux Kernel Networking: Implementation and Theory*. Apress.
2. Menage, P. (2007). Adding Generic Process Containers to the Linux Kernel. In *Proceedings of the Linux Symposium* (Vol. 2, pp. 45-57).
3. Snyk Security Research. (2018). *Zip Slip Vulnerability: Arbitrary File Overwrite Through Directory Traversal*. Available at: https://snyk.io/research/zip-slip-vulnerability .
4. Amazon Web Services. (2024). *AWS Well-Architected Framework: Security Pillar*. AWS Whitepapers.
5. Open Web Application Security Project (OWASP). (2023). *Top 10 API Security Risks*. OWASP Foundation. Available at: https://owasp.org/API-Security/ .
6. Docker Documentation. (2024). *Runtime Privilege and Linux Capabilities in Docker Engine*. Docker Inc. Available at: https://docs.docker.com/engine/security/ .
7. Ramirez, S. (2024). *FastAPI: Modern, High-Performance Web Framework for Python*. Available at: https://fastapi.tiangolo.com .
8. Bayer, M. (2024). *SQLAlchemy: The Database Toolkit for Python*. Available at: https://www.sqlalchemy.org .
9. Amazon Web Services. (2024). *AWS CloudFormation User Guide: Resource Specification and Best Practices*. AWS Documentation.
10. Sysoev, I., & Nginx Authors. (2024). *Nginx Web Server Documentation: Rate Limiting and Reverse Proxy*. F5 Inc. Available at: https://nginx.org/en/docs/ .
11. You, E., & Vite Contributors. (2024). *Vite: Next Generation Frontend Tooling*. Available at: https://vite.dev .
12. PostgreSQL Global Development Group. (2024). *PostgreSQL 15 Documentation: Reliability and Data Durability*. Available at: https://www.postgresql.org/docs/15/ .